Tech Notes Day 2: Core Activities

2.1 Sequence Detector (Moore FSM)

- **Design Approach:** Implemented a 5-state Moore machine to detect the sequence "1011". An overlap transition was explicitly coded in State 4 to route back to State 1 or State 2 depending on the input, ensuring overlapping sequences are caught.
- **Verification Steps:** Injected the standard "1011" sequence into the testbench, followed immediately by "011" to create the overlapping scenario "1011011".
- **Results & Issues:** The FSM correctly asserted the output twice for the overlapping input stream.

2.2 Vending Machine Controller

- **Design Approach:** Designed an FSM relying on a balance register to track 5 and 10 rupee coin insertions. Included default assignments for dispense, return_change, and refund at the top of the combinational block to strictly prevent inferred latches.
- **Verification Steps:** Tested a 20-rupee overpayment scenario (two 10-rupee coins) to verify simultaneous dispense and return_change assertions, and a cancellation scenario (inserting 5 then hitting cancel) to verify the refund logic.
- **Results & Issues:** The design simulated flawlessly without generating any latch warnings.

2.3 Synchronous FIFO

- **Design Approach:** Constructed a parameterized FIFO (depth 16, width 8) using read (rp) and write (wp) pointers alongside a count register. Explicitly handled the simultaneous read-and-write condition by incrementing both pointers while holding the count stable, avoiding state corruption.
- **Verification Steps:** (Based on RTL structure) Verified boundary conditions for full (count == DEPTH) and empty (count == 0) flags.
- **Results & Issues:** Pointer wrap-around and simultaneous read/write operations behave correctly based on the case logic.

2.4 FSM and FIFO Design Review

Overview: Following the initial drafting of the Day 2 FSM controllers and the synchronous FIFO, I conducted a self-review of the RTL code to identify potential synthesis issues, specifically looking for inferred latches, incomplete conditional branches, and unhandled states.

Issues Found and Fixes Applied:

- **Vending Machine Controller (Inferred Latches):**
    - *Issue:* In the initial draft of vending.v, the combinational always @(*) block did not assign a value to dispense, return_change, and refund in every possible execution path (e.g., when no coins were inserted). This would cause the synthesis tool to infer unwanted latches to hold the previous values.
    - *Fix:* I added default assignments (dispense = 0;, return_change = 0;, refund = 0;, and next_balance = balance;) at the very top of the combinational block. This guarantees that all outputs have a defined state every cycle, strictly preventing latches.
- **Sequence Detector (Missing Default Case):**
    - *Issue:* The Moore FSM in seq_moore.v uses a 3-bit state register, which has 8 possible states. However, the sequence detector only utilizes 5 valid states (0 through 4). Initially, states 5, 6, and 7 were left undefined in the case statement.
    - *Fix:* I added a default: next = 3'd0; branch to the state transition logic. This ensures that if the FSM ever enters an illegal state due to a glitch, it will safely reset back to the initial state rather than locking up.
- **Synchronous FIFO (Simultaneous Read/Write Collision):**
    - *Issue:* When reviewing the FIFO control logic, I realized that if a read and a write occurred in the exact same clock cycle (we = 1 and re = 1), the count register would be subjected to simultaneous increment and decrement commands, potentially corrupting the depth tracker.
    - *Fix:* I modified the control logic to use a concatenated case statement checking {we & ~full, re & ~empty}. I explicitly added a 2'b11 case where both the read and write pointers increment, but the count register is explicitly bypassed and remains unchanged.
- **Traffic Light Controller (Incomplete Reset):**
    - *Issue:* During the review of traffic.v, I noticed the asynchronous pedestrian request flag (ped_pending) was not being cleared during the system reset.
    - *Fix:* I updated the if (rst) block to explicitly force ped_pending <= 0; alongside the state and counter resets, ensuring a completely clean initialization.

- **ATM Controller (Incomplete if-else branches):**
    - *Issue:* In the ATM state machine, several conditional transitions (like checking if the PIN is correct) did not have an `else` branch specifying what happens to internal registers if the condition is false, which could lead to latches.
    - *Fix:* I added default assignments at the very top of the combinational `always @(*)` block (`next_state = state; next_balance = balance; next_attempts = pin_attempts;`). This ensures all registers maintain their current state safely unless explicitly overridden by the `if` conditions.

Tech Notes Day 2: additional activities

2.5 Traffic Light with Pedestrian Request

- **Design Approach:** Built a time-based FSM with an internal down-counter to control Green (5 cycles), Yellow (2 cycles), Red (5 cycles), and Walk (4 cycles) phases. Used a ped_pending flag to safely capture asynchronous pedestrian requests occurring during Green or Red phases.
- **Verification Steps:** Asserted the ped_req signal during the Green phase. Verified that the FSM waited for the counter to expire, transitioned through Yellow, and then routed to the Walk state rather than standard Red.
- **Results & Issues:** Pedestrian interrupts are handled safely without cutting the active Green or Yellow phases short.

2.6 ATM Controller

- **Design Approach:** Implemented a 4-state Mealy-Moore hybrid FSM (IDLE, PIN_WAIT, TRANSACT, LOCKED). The controller uses an internal pin_attempts counter that resets upon returning to IDLE and increments upon incorrect PIN entries. The FSM defaults to a locked state if the attempts counter reaches two (the third attempt). Outputs like dispense and reject_funds are evaluated combinationally in the TRANSACT state based on the requested amount versus the current balance.
- **State Diagram (Textual Representation):**
    - **IDLE** → (card_insert) → **PIN_WAIT**
    - **PIN_WAIT** → (pin_enter & pin_in == 1234) → **TRANSACT**
    - **PIN_WAIT** → (pin_enter & pin_in != 1234 & attempts < 2) → **PIN_WAIT**
    - **PIN_WAIT** → (pin_enter & pin_in != 1234 & attempts == 2) → **LOCKED**
    - **TRANSACT** → (card_remove) → **IDLE**

- **Verification Steps:** Programmed three distinct scenarios in the testbench: a successful ₹2000 withdrawal updating the initial ₹5000 balance, an attempted withdrawal of ₹4000 triggering a reject_funds flag, and a sequence of three consecutive incorrect PINs triggering the locked state.
- **Results & Issues:** All outputs responded correctly to the input sequences. I had to explicitly include a card_remove signal to safely reset the state machine back to IDLE between the different test scenarios without hard-resetting the internal account balance.

2.7 Round-Robin Arbiter

- **Design Approach:** Implemented a fair access scheme for 4 requesters using a turn register. The logic grants access in a rotating modulo-4 sequence and intentionally holds the grant active until the current requester pulls its request low.
- **Verification Steps:** (Based on RTL structure) Monitored the turn variable under constant requests to ensure it cycles fairly from 0 to 3 without skipping or starving any requester.
- **Results & Issues:** The arbiter successfully locks onto a granted state and successfully shifts priority upon release.

2.8 RAM, ROM, and Watchdog Timer

- **Design Approach:** Designed the RAM as a synchronous-write, asynchronous-read memory array. The ROM was synthesized using an initial $readmemh("rom.hex", mem) block. The Watchdog Timer was built as a 4-bit counter that asserts a wdt_rst pulse upon reaching 15 cycles unless cleared by a kick signal.
- **Verification Steps:** Wrote 8'hAA to RAM address 5 and read it back. Let the WDT run without kicking to verify the reset assertion at cycle 15, then pulsed the kick signal to verify the counter resets to 0.
- **Results & Issues:** All memory transactions executed correctly, and the WDT behaved as an effective timeout monitor.